57

分布式系统安全

本章作者：**Peter Reiher（UCLA）**

57.1 引言

操作系统只能控制它自己那台机器上的资源。因此，在需要多台机器协作的分布式系统中，操作系统要提供安全会遇到难题。主要有两大问题：

- 分布式系统中的其他机器未必会正确执行你想要的安全策略，它们甚至可能是冒充可信伙伴的对手。我们无法控制远程系统，却仍然必须能够信任它们交给我们的凭证和能力是有效的。
- 分布式系统中的机器要通过网络通信，而这个网络没有哪台机器能完全控制，一般来说也不可信。对手往往和我们一样能接入这个网络，可以伪造、复制、重放、篡改、销毁和延迟我们的消息，总之能干扰我们对网络的使用。

前面说过，密码学是我们在这里要用的主要工具，但我们也说过，密码学很难用对。这么看来，这里正适合使用精心设计的标准工具，而不是指望每个人都自己造一套。完全正确。于是：

> 关键问题：如何保护分布式系统中的操作
> 跨越多台机器的系统，怎样才能保证它的安全？有哪些工具可以帮助我们保护这类系统？怎样正确地使用它们？使用这些工具时，有哪些地方需要我们格外小心、仔细斟酌？

57.2 身份认证的作用

分布式系统中的伙伴会不会执行我们的安全策略，我们无法确定，这种不确定性该怎么应对？大多数情况下，我们做不了什么。充其量只能设法就策略达成一致，然后指望大家都遵守约定。在少数特殊情况下，我们能拿到高质量的证据，证明伙伴的行为是正确的，但一般来说这并不容易。比如，我们怎么知道对方用了全盘加密？怎么知道对方已经仔细清除了我们不再使用的某个加密密钥？又怎么知道对方给自己文件的本地副本正确设置了访问控制？他们可以说自己做了，但我们怎么知道是真是假？

一般来说，我们没法知道。不过你对此已经习惯了。在现实生活中，你的朋友和亲戚知道你的一些秘密，他们可能有你家的钥匙；你把车借给他们，也相当确定能把车要回来。这

并不是因为你有什么完美的机制能阻止这些可信的人做坏事，而是因为你相当确定他们不会做。如果你看错了人，或许你能发现他们做了不该做的事，然后采取补救措施（比如换锁，或者报警说车被偷了）。在分布式计算机系统中，我们也要依靠类似的办法。我们只能相信有些参与方会规规矩矩。有些情况下，我们能发现他们没守规矩，于是相应地调整对他们的信任，也许还会采取其他补救措施。

当然，在网络世界里，我们的行动都是隔着网络远距离进行的，我们能看到的，只有在网络上进进出出的比特。要让基于信任的方案行得通，我们必须非常确定：我们发出去的比特，伙伴能够验证它们确实来自我们；而收到的比特，也确实是伙伴生成的。这就是身份认证的工作。正如前面身份认证一章所说，在网络上工作时，我们只能根据一串比特来认证身份。最常用的是基于“你*知道*什么”的认证方式。现在回想一下前面几章：运行在远程操作系统上的某个东西，可能知道哪些别人都不知道的东西？密码怎么样？私钥呢？

分布式系统中的身份认证大多依靠这两样东西之一。要么你要求远程机器向你提供密码，要么你要求它用一把只存放在那台机器上的私钥来提供证据[1]。无论哪种情况，你都需要知道某些东西才能核验身份：要么是密码（更好的做法是密码加盐后的密码学散列值），要么是公钥。

这两种方式分别适合什么场合？如果有大量参与方需要向同一个参与方证明自己的身份，密码往往比较好用。如果是一个参与方需要向大量参与方证明自己的身份，公钥往往比较好用。为什么？用密码认证时，认证结果证明的是有人知道这个密码。如果你想确切知道这个人是谁（这通常很重要），就只能让被认证的一方和核验的一方知道这个密码。而用公钥时，许多参与方都可以知道公钥，但只有知道对应私钥的那一方才能证明自己的身份。所以这两种机制我们都会用，只是用在不同的场合。网站向用户证明自己的身份时，用的是公钥密码学。网站只要分发同一把公钥（给大量用户），所有用户就都能认证它，不必为了向每个用户证明自己而分别保存一份认证信息。用户向网站证明自己的身份时，用的是密码。每个用户都必须单独向网站认证，因此我们需要每个用户都有一份独一无二的身份信息，最好还方便人来使用。建立和分发公钥很难，而设置各自的密码相对容易。

在分布式系统中，这两种认证机制实际上怎么用？如果我们想让远程伙伴通过密码证明自己的身份，就会要求它把密码交给我们核验。这样做的话，密码在网络上传输时必须加密；否则，任何在网络上窃听的人（对许多无线网络来说，窃听很容易）都能轻而易举地拿到明文发送的密码。要加密密码，我们必须事先已经有一把共享的对称密钥，或者有伙伴的公钥。下面我们先集中讨论怎样拿到这把公钥：要么直接使用它，要么借助它建立起加密，保护传输中的密码。

本章余下的部分都会讨论怎样保护网络连接的安全。但请不要忘了，即使你把网络保护得滴水不漏，网络另一端那个你无法控制的站点，依然是重大的安全挑战。如果你的伙伴已经被攻破，转而攻击你，那么即使这次攻击经过了认证和加密，对你也没什么安慰。

[1]我们偶尔也会用其他方法，比如智能卡或远程生物识别读取器。不过在今天的系统中，这些方法不太常见。如果你理解了在分布式系统身份认证中怎样使用密码和公钥密码学，大概也就能想明白怎样正确使用其他这些技术。如果你还不理解，最好先弄懂常用的技术，再去研究不常用的。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

57.3 分布式系统中的公钥认证

公钥不需要保密，但我们需要确定它真的属于我们的伙伴。如果能当面见面，伙伴可以直接以某种形式把公钥交给我们，这时我们可以相当确定它就是正确的那一把。不过这种做法局限性很大，因为我们打交道的伙伴往往从未见过面。说到底，Amazon[2]或 Google 的“面”又是谁的呢?

幸运的是，既然公钥不需要保密，我们就可以干脆生成一串包含公钥的比特。谁拿到这串比特的副本，谁就有了这把公钥。可是他们怎么确定这把公钥属于谁? 假如有另一个可信的第三方，所有需要认证我们伙伴的人都认识它，它用自己的私钥对这串比特做密码学签名，证明这串比特确实属于我们的伙伴，结果会怎样? 只要我们能核验这个签名，就可以确定这串比特确实代表伙伴的公钥，至少在我们信任这个签名的第三方的范围内是如此。

我们在因特网上认证网站和许多其他实体，实际用的正是这种技术。每次浏览网页或进行其他任何基于 Web 的活动，你都在使用它。这串经过签名的比特叫作**证书**（certificate）。它基本上包含拥有这把公钥的一方的信息、公钥本身以及其他信息，比如有效期。包括公钥在内的这一整套信息先经过密码学散列运算，再用可信第三方的私钥加密散列结果，这样就对证书做了数字签名。只要你拿到证书的副本，又能核验签名，就能得知某人的公钥，即使你从未见过对方，也从未和对方直接打过交道。从某些方面看，这是一项支撑起整个因特网的美妙技术。

我们简单地过一个例子，把这些概念理清楚。假设 Frobazz 公司想为自己的公钥 K_F 申请一张证书。Frobazz 公司花了大价钱请 Acmesign 公司签发一张证书。Acmesign 是一家广受信任的公司，业务就是出售证书。这类公司通常叫作**证书颁发机构**（**Certificate Authorities**，**CA**），因为它们签发的权威证书受到许多参与方的信任。Acmesign 会核查 Frobazz 公司，确保来申请证书的人确实是 Frobazz 的合法代表。接着，Acmesign 会非常非常仔细地确认，它要嵌入证书的那把公钥确实是 Frobazz 想用的那一把。确认无误后，Acmesign 对 Frobazz 的名称、公钥 K_F 以及其他信息运行一个密码学散列算法（可能是 SHA-3，截至 2020 年它还没有被破解，SHA-1 则已经被破解），得到散列值 H_F。然后，Acmesign 用自己的私钥 P_A 加密 H_F，得到数字签名 S_F。最后，Acmesign 把用来生成 H_F 的全部信息，加上 Acmesign 自己的身份和签名 S_F，合成证书 C_F，交给 Frobazz，想必是收了钱的。记住，C_F 只不过是一些比特而已。

现在，Frobazz 公司想通过因特网向它的一位顾客证明自己的身份。如果顾客已经有 Frobazz 的公钥，我们可以直接使用公钥认证机制。如果顾客没有这把公钥，Frobazz 就把 C_F 发给顾客。顾客检查证书，看到它是 Acmesign 用（比如说）SHA-3 生成的，于是把 Acmesign 当初做散列的那些信息（全都在证书里）同样用 SHA-3 运算一遍，得到 $H_{F'}$。然后，顾客用 Acmesign 的公钥解密 S_F（它也在证书里），得到 H_F。如果一切正常，H_F 就等于 $H_{F'}$，顾客就知道证书里的公钥确实属于 Frobazz。接下来就可以进行基于公钥的认证了[3]。如果两个散列值不完全相同，顾客就知道其中有鬼，不会接受这张证书。

[2]如果 Jeff Bezos 得亲自登门拜访每一位顾客，把 Amazon 的公钥交给他们，Amazon 还能有多成功? 答案是：不会那么成功。

[3]而且确实必须进行，因为核验证书这一大套工作，只是告诉了顾客 Frobazz 的公钥是什么，并不能向顾客保证发来证书的一方确实就是 Frobazz，或者知道 Frobazz 的私钥。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

用这种方式获取公钥，有几个很好的特性。第一，在顾客核验证书的过程中，签名机构（在我们的例子里是 Acmesign）不需要参与。实际上，Frobazz 也不需要参与。顾客可以从任何地方拿到这张证书，对其有效性的把握都一样。第二，每位顾客只需要做一次。拿到证书并核验之后，顾客就有了需要的公钥。从此以后，顾客只要把公钥保存起来使用就行。如果公钥因为某种原因丢了，顾客可以从证书里再提取一次（如果证书还保存着），或者把获取证书的过程再走一遍。第三，在通过核验证书证明对方身份之前，顾客不必信任自称是 Frobazz 的那一方。证书核验通过之前，顾客可以谨慎行事。

如果你认真读了前面几章，现在应该在心里嘀咕："等一下，这里是不是有个先有鸡还是先有蛋的问题？"我们要通过获取 Frobazz 的证书来得知它的公钥。证书由 Acmesign 签名。我们要靠 Acmesign 的公钥来核验签名。那 Acmesign 的公钥又是从哪儿来的？我们真心希望你刚才挠了挠头，问了自己这个问题，因为如果你问了，就说明你理解了因特网身份认证问题的本质。归根结底，我们必须自举。你得想方设法先拿到某个你信任的人的公钥。一旦拿到了，如果它确实是正确的一方的正确公钥，你就能借此获得许多其他公钥。但如果没有一个起点，你几乎什么也做不了。

这把最初的公钥从哪里来？最常见的是来自你获取并安装的某个软件。你最常用的大概是浏览器，它通常自带几百家可信机构的公钥[4]。每当你访问一个注重安全的新网站，它会给你一张证书，里面有这个网站的公钥，签名者是浏览器里预先配置好的某家可信机构。你用这家机构预先配置好的公钥核验证书是否合法，之后就知道了这个网站的公钥。从此以后，你可以用网站的公钥来认证它。这里还有一些重要的注意事项（也有一些解决这些问题的有趣办法），不过我们暂且把它们放到一边。

任何人都可以创建证书，不只是那些受信任的 CA。你可以向以签发证书为业的机构申请一张，也可以按照某个证书标准从头自己创建一张（X.509 是最常用的证书标准 [I12]）。必要的条件是：被认证的一方和进行认证的各方，都必须信任创建证书的那一方。如果他们不信任那一方，又凭什么相信证书是正确的呢？

如果你在搭建自己的分布式系统，可以在一台你（以及系统中其他参与者）信任的机器上创建自己的证书，然后把这台签发证书的机器的公钥小心地手工安装到所有需要它的地方，以此解决自举问题。现在已经有不少用来创建证书的软件包，而且和其他关键的密码学软件一样，你最好使用现成的、可信的实现，而不要自己写一个。你也许可以看看 PGP（既有提供支持的商业版本，也有兼容但支持较少的免费版本）[P16]，当然还有别的选择。如果你面对的机器数量固定，又能以某种合理的方式手工分发公钥，那就完全可以不用证书。记住，公钥证书唯一的用处就是分发公钥，所以如果你的公钥已经在它们该在的地方了，就不需要证书。

好了，不管用什么办法，你已经拿到了认证某台远程机器所需的公钥。接下来呢？嗯，对方用私钥加密后发给你的任何东西，都只能用对方的公钥解密，所以凡是用这把公钥能正确解密的东西，一定都来自对方，对吧？没错，它在某个时候确实来自对方。但对手可能复制了这个站点过去某个时候发出的一条合法消息，然后在以后的某一天再发一遍。根据具体情况，这可能会惹出麻烦，因为你可能会根据这条消息采取行动，而那并不是合法站点要求

[4]你确实知道外面有几百家公司，你在网上做的每一件事都托付给了它们，对吧？好吧，不管你知不知道它们，实际上你都在这种程度上信任着它们。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

的。所以我们通常会采取措施，确保自己不会遭受**重放攻击**（replay attack）。这类措施一般是确保每条加密消息都包含其他任何消息里都没有的独特信息。标准的密码学协议都已经正确地内置了这一特性，所以只要你听从我们的建议，使用其中一种，就能防范这类重放攻击。如果你坚持自己构建密码学方案，就得对这个问题有更多的了解，还要非常小心地运用这些知识。此外，公钥密码学开销很大。我们希望尽快停止使用它，同时又希望继续得到身份认证的保证。讨论 SSL 和 TLS 时，我们会看到怎样做到这一点。

57.4 分布式系统中的密码认证

在分布式系统中，另一种常用的认证方式是使用密码。前面说过，这种方式最适合的情况是，任何一个特定的密码都只涉及两方：被认证的一方和进行认证的一方。单个用户向一个托管着许多用户的站点证明自己的身份时，比如你登录 Amazon 时，用密码是合理的。而站点向单个用户证明自己的身份时，比如一个自称是 Amazon 的网站想和你做生意时，用密码就不合理了。这种情况下，公钥认证的效果更好。

在适合使用密码的场合，怎样在网络上正确地处理密码认证?密码通常与某个特定的用户 ID 关联，因此用户要把这个 ID 和密码提供给要求认证的站点。这一般要经过网络，而我们通常无法保证网络能提供机密性。如果我们的密码泄露给了别人，别人就能冒充我们，所以我们必须给这种跨网络的认证加上机密性保护，一般至少要加密密码本身（不过把涉及的所有内容都加密更好）。因此，Alice 向 Frobazz 公司的网站证明自己身份的典型过程是：网站索要用户 ID 和密码，Alice 提供这两样东西，但在通过网络发送之前先把它们加密。

你应该会问一个明显的问题：用哪把密钥加密? 嗯，如果 Frobazz 已经像上面讨论的那样用公钥向 Alice 证明了自己的身份，Alice 就可以用 Frobazz 的公钥加密自己的用户 ID 和密码。Frobazz 公司有对应的私钥，能够核验它们，而其他人都读不了。实际上，出于各种原因，光这样做还不够，其中就包括上面提到的重放攻击。但我们确实可以用 Frobazz 的私钥来建立加密，保护传输中的 Alice 的密码，实际上也正是这么做的。具体细节我们会在 SSL/TLS 一节讨论。

我们在身份认证一章讨论过密码的选择和管理问题，那些内容在网络环境中同样适用。除此之外，关于怎样使用密码就没有太多要说的了，只想指出一点：远程站点核验过密码之后，它实际上知道了什么? 它知道发来密码的站点或用户知道这个密码，并且在密码强度所能保证的程度内，可以认为这个站点或用户就是它自称的那一方。可是以后再收到据称来自这个站点的消息呢? 记住，任何人都能随意创建任何消息，所以如果我们只是核验了远程站点发来的密码是对的，我们就只知道那一条消息是真实的。我们不想在发出的每条消息里都附上密码，就像我们不想用公钥去加密发出的每条消息一样。我们会用这两种认证技术建立最初的真实性，再用别的东西把这份最初的真实性和后续的交互绑在一起。我们直接进入 SSL/TLS，讨论怎样做到这一点。

57.5 SSL/TLS

我们在前面某一章看到，在现代系统中，进程之间通信的标准方法是套接字。进程位于不同机器上时也是如此。所以，要给穿越无保护网络的通信加上密码学保护，一个很自然的

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

办法就是给套接字加上密码学功能。这正是很多年前设计 **SSL**（**Secure Socket Layer**，安全套接字层）的目的。遗憾的是，SSL 并没有完全做对。这是因为要做对实在太难了，并不是设计和构建它的人粗心大意。他们从错误中吸取了教训，又设计了一个新版本的加密套接字，叫作**传输层安全**（**Transport Layer Security**，**TLS**）[5]。你会经常听人说在用 SSL，他们通常是把 SSL 当作 SSL/TLS 的简称。正式地说，SSL 是不安全的，任何场合都不应该再用它。请使用 TLS。唯一的例外是，有些非常老的设备上运行的软件可能不支持 TLS。在这种情况下，用 SSL 总比什么都不用好。从这里开始，我们也和别人一样采用这个简称，因为大家都这么叫。

SSL 背后的概念很简单：让加密过的数据通过一个普通的套接字传输。你建立一个套接字，再建立一个专门的结构来执行你想要的密码学运算，然后把这个结构的输出接到套接字的输入上。在另一端，把这个过程反过来。概念上很简单，执行起来却相当繁琐，要经过好几个步骤才能达到想要的结果。由于 SSL 本身的通用性，还会有更多复杂之处。这项技术的设计目标是支持各种密码学操作和许多不同的密码算法，还支持发送方和接收方之间用多种方法进行密钥交换和身份认证。

给程序加上 SSL 的过程很复杂，需要使用特定的库，并按一定顺序调用这些库，才能建立一个正确的 SSL 连接。我们不会在这里一步步地讲解这些操作，但要正确使用 SSL，你需要去学习它们。这些操作的目的，大体上是在 SSL 支持的密码学选项方面，以及你在程序中使用这些选项的方式方面，都提供很大的通用性。例如，如果你需要的话，可以通过这些设置调用，让一组 SSL 连接使用 AES-128，另一组使用 AES-256。

建立 SSL 连接有一个常见的要求，我们会稍微详细地讲一讲，那就是怎样安全地分发你要为正在建立的连接使用的密码学密钥。密码学的最佳实践要求，你建立的每个连接都用一把全新的密钥来加密大部分数据。公钥/私钥你会用来做很多次身份认证，但正如前面讨论的，认证了伙伴之后，你需要用对称密码学来加密数据，而且你需要一把新鲜的密钥。即使你和同一个伙伴同时建立了多个 SSL 连接，每个连接也都需要一把不同的对称密钥。

那么，建立一个新的 SSL 连接需要做些什么？我们不会把所有繁琐的细节都过一遍，但本质上，在没有可用的对称密钥时，SSL 需要（通常）基于非对称密码学自举出一个安全的连接。（在讨论 SSL 时，你会经常听到“通常”“一般”和“默认”这类字眼，因为 SSL 能支持的选项非常多，而其中大多数一般都不是你想用的。）第一步是在客户端和服务器之间开始协商。每一方可能只能处理特定的密码算法、安全散列、密钥分发策略或身份认证方案，这取决于它安装的 SSL 版本、配置方式，以及两边建立 SSL 连接的程序是怎么写的。在最常见的情况下，协商的结果是双方找到一组都能接受的密码算法和技术，在安全性和性能之间取得平衡。例如，它们可能用 2048 位密钥的 RSA 做非对称密码学，用某种形式的 Diffie-Hellman 密钥交换机制（参见关于这个机制的补充）建立一把新的对称密钥，用 SHA-3 生成保证完整性的安全散列，再用 256 位密钥的 AES 做大批量加密。一个现代的 SSL 安装可能支持 50 种甚至更多不同的选项组合。

[5]实际上，就连 TLS 最初的几个版本也没有完全做对。截至 2020 年，TLS 的当前版本是 1.3，你大概应该用这个版本。TLS 1.3 修补了 TLS 1.2 存在的一些漏洞。SSL/TLS 一再需要修改的历史，应该能进一步让你认识到正确使用密码学有多难，而这又应该促使你发誓永远不要尝试自己写一套密码学方案。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

补充：Diffie-Hellman 密钥交换

如果你想在两方之间共享一把秘密密钥，可它们只能通过不安全的信道通信，窃听者能听到它们说的任何话，那该怎么办？你可能觉得这个问题根本无解，但你错了。两位极其聪明的密码学家 Whitfield Diffie 和 Martin Hellman 多年前就解决了这个问题，他们的解法如今被广泛使用。它叫作 **Diffie-Hellman 密钥交换**（Diffie-Hellman key exchange）。

它的原理是这样的。假设 Alice 和 Bob 想共享一把秘密密钥，但目前除了能互相发消息之外，没有任何共享的东西。首先，他们商定两个数：n（一个大素数）和 g（模 n 的原根）。这一步可以通过不安全的信道进行，因为 n 和 g 不需要保密。Alice 选一个大随机整数，比如 x，计算 $X = g^x \bmod n$，然后把 X 发给 Bob。Bob 独立地选一个大随机整数，比如 y，计算 $Y = g^y \bmod n$，然后把 Y 发给 Alice。窃听者能听到 X 和 Y，但由于 Alice 和 Bob 没有发送 x 或 y，窃听者并不知道这两个值。Alice 和 Bob 必须对 x 和 y 保密，这一点很重要。

现在，Alice 计算 $k = Y^x \bmod n$，Bob 计算 $k = X^y \bmod n$。Alice 和 Bob 通过这两个计算得到的是同一个 k 值。为什么？因为 $Y^x \bmod n = (g^y \bmod n)^x \bmod n$，它又等于 $g^{yx} \bmod n$。而 $X^y \bmod n = (g^x \bmod n)^y \bmod n = g^{xy} \bmod n$，和 Alice 得到的结果一样。这里没什么魔法，幂运算和取模运算本来就是这样的。啊，数学多么美妙！所以两边算出的 k 相同，Alice 和 Bob 都知道它。

那些窃听者呢？他们知道 g、n、X 和 Y，但不知道 x 或 y。他们可以算出 $k' = XY \bmod n$，但这并不等于 Alice 和 Bob 算出的 k。他们确实有办法推导出 x 或 y，从而得到足以算出 k 的信息，但这些办法要么要求他们对 n 的每一个可能取值都做一次计算（这就是 n 要非常大的原因），要么要求他们计算离散对数。离散对数是可以求解的，但对很大的数来说，在计算上不可行。所以，只要素数 n 足够大（并满足其他一些性质），窃听者就无计可施。要多大？600 位（十进制）的素数应该就够了。

很巧妙，对吧？但在网络上使用 Diffie-Hellman 时，有一个美中不足之处。它能确保你和某人安全地共享一把密钥，却无法保证你是在和谁共享。也许 Alice 正如她所想、所希望的那样在和 Bob 共享密钥，但也可能她是在和冒充 Bob、注入了自己的 Y 的 Mallory 共享密钥。我们通常很在意自己在和谁进行安全通信，所以一般会给 Diffie-Hellman 加上一个身份认证机制，确保伙伴的身份。

有些情况下，指定这许多组合中哪些对你的系统来说是可以接受的，可能很重要；但很多时候，大多数组合都行，这时你可以让 SSL 为每个连接自行决定用哪一种，你自己不用操心。协商会在你看不见的地方进行，然后 SSL 就开始干正事：至少认证服务器（客户端可选），创建并分发一把新的对称密钥，再用这把密钥通过选定的密码算法进行通信。

我们可以用 Diffie-Hellman 密钥交换来创建密钥（SSL 也经常这么做），但我们必须确定自己是在和谁共享这把密钥。SSL 为此提供了多种办法。最常见的方法是，客户端拿到一张包含服务器公钥的证书（通常是由服务器发给客户端），然后用证书里的公钥核验服务器消息的真实性。客户端也可以通过其他途径获得证书，不过不太常见。注意，由服务器发送

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

证书和客户端通过其他途径获取证书，安全程度完全一样（不安全的程度也一样）。证书的安全性并不取决于用什么方法传输它，而取决于嵌在证书里的密码学内容。

有了证书（不管客户端是怎么拿到的），Diffie-Hellman 密钥交换就可以在认证的前提下进行了。服务器会用自己的私钥对它的 Diffie-Hellman 消息签名，这样客户端就能确定，这次密钥交换中的伙伴是正确的服务器。客户端通常不提供（甚至根本没有）自己的证书，所以无法对自己的 Diffie-Hellman 消息签名。这意味着，SSL 的 Diffie-Hellman 密钥交换完成后，客户端通常相当确定服务器是谁，服务器却对客户端的身份一无所知。（同样，并不是 SSL 的所有用法都如此。SSL 也有一些建立连接的选项，让双方都知道对方的公钥，密钥交换在两边都经过认证。只是这些选项不是最常用的，尤其不是保护网页浏览时通常使用的那些。）

回想本章前面的讨论，很多情况下，此时服务器不确定客户端的身份其实并不是问题。前面说过，客户端大概会想用密码来证明自己的身份，而不是用从证书里提取的公钥。只要服务器在拿到并核验客户端的密码之前，不允许客户端做任何需要信任的事，服务器其实根本不在乎客户端是谁。许多服务器会向匿名客户端提供一些服务（比如向它们提供公开的信息），所以只要它们在进入更敏感的内容之前能从客户端拿到密码，就不存在安全问题。服务器可以在 SSL 连接建立后的任何时候，再向客户端索要用户 ID 和密码。由于建立 SSL 连接时已经设置了一把对称密钥，ID 和密码的交换就可以用这把密钥保护。

关于 SSL/TLS，最后再说一句：它是一个协议，而不是一个软件包。实现这个协议的软件包有好几种。理想情况下，只要它们都正确地实现了协议，彼此就都能正确交互。不过，它们实现协议的代码各不相同。因此，SSL/TLS 某个实现中的软件缺陷，在其他实现中未必存在。例如，心脏出血（Heartbleed）攻击利用的是 OpenSSL 的实现细节 [H14]，而在其他实现中并不存在，比如 Microsoft 的 Windows 操作系统中的 SSL/TLS 版本。另一方面，当前的 SSL/TLS 协议定义本身也可能包含协议缺陷，任何遵守协议的实现中都会有。如果你听说某个与 SSL 有关的安全问题，在进一步行动之前，先弄清楚它是协议缺陷还是实现缺陷。如果是实现缺陷，而你用的又是另一种实现，你可能就不需要采取任何应对措施。

57.6 其他身份认证方法

虽然密码和公钥是认证远程用户或机器最常用的方式，但还有其他选择。其中一种一直都在用。像上面描述的那样，你通过提供密码向某个网站证明了自己的身份之后，这个网站会一直认为这次认证有效。你每次点击链接，或者和它进行其他交互时，它都不会再要你的密码。（这样也好。想象一下，如果每做一件事都得提供一次密码，那该多烦人。）如果此时你的会话是加密的，网站可以把你正确使用密码学这件事当作一种身份认证；但你甚至可能退出浏览器、重新启动、再回到那个网站，依然被当作已认证的用户，而不会再被要求输入密码。到了这时，你用的已经不是之前那套密码学了，因为你已经建立了一个新会话，设置了一把新的密码学密钥。你的伙伴是怎么认证收到新密钥的一方就是你的?

在这种情况下，与你打交道的站点选择了在安全性上做一个权衡。它过去某个时候用你的密码核验了你的身份，之后就依靠另一种方法来认证你。一种常用的方法是使用 **Web Cookie**（web cookies）。Web Cookie 是网站发给客户端的一些数据，网站希望客户端把这些数据保存起来，下次再和服务器通信时一并发回去。大多数浏览器都内置了 Web Cookie，处

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

理它们是在后台进行的，不需要用户干预。如果正确使用密码学，核验过客户端密码的服务器就可以创建一个安全地保存了客户端身份的 Web Cookie。客户端再次与服务器通信时，浏览器会自动在请求中附上这个 Cookie，服务器借此就能核验客户端的身份，而不必再次索要密码[6]。

如果你花几分钟想一想这种认证方法，可能会想到一些相关的安全问题。设计这项技术的人已经解决了其中一些问题，比如防止窃听者直接使用在网络上传输时被复制的 Cookie。然而，还有一些安全问题（比如合法用户以外的人使用了运行浏览器、保存着 Cookie 的那台计算机）是这类 Cookie 解决不了的，而如果要求用户每次都提供密码，这些问题本来是可以解决的。构建自己的系统时，你需要自己考虑这类安全权衡。是只在绝对必要时才要求输入密码，让用户用起来更省事好，还是频繁要求用户证明身份、给用户更高的安全性好？重点不在于这个问题只有一个正确答案，而在于设计系统时你需要考虑这类问题。

还有其他身份认证方式。一个例子是所谓的**挑战/应答协议**（challenge/response protocol）。远程机器给你发来一个挑战，通常是一个数。要证明自己的身份，你必须对这个挑战执行某种运算，得出一个应答。这种运算应该只有真正的那一方才能完成，所以它大概要依赖一个只有那一方知道、别人都不知道的秘密。把这个秘密作用于挑战，就得到应答，再把应答发给服务器。服务器必须能够核验对方给出的应答是否正确。每次发出的挑战都不同，要求的应答也不同，所以攻击者窃听并记下旧的挑战和应答也占不到任何便宜。因此，挑战和应答都不需要加密。挑战/应答系统通常会对挑战加上秘密做某种密码学运算，也许是一次散列运算，来生成应答。这类运算由机器来做比由人来做更合适，所以要么由你的计算机替你算出应答，要么你有一个专门的硬件令牌来处理。无论哪种方式，挑战/应答系统都要求发起挑战的机器和要证明自己身份的机器事先做好安排。硬件令牌或数据秘密必须在发出挑战之前就已经设置并分发好。

另一种身份认证方式是使用认证服务器。本质上，你和一台你信任、也信任你的服务器通信。你想向其证明身份的那一方也必须信任这台服务器。认证服务器会以某种安全的形式（通常涉及密码学）为你的身份担保。需要认证你的那一方能够核验认证服务器提供的安全信息，从而确定服务器已经核验过你的身份。由于你想与之通信的那一方信任认证服务器，它现在也就相信你就是你自称的那个人。粗略地说，证书和 CA 是这类认证服务器的离线版本。还有更主动的在线版本，需要两台想要通信的机器与一台或多台认证服务器之间进行各种网络交互。和 CA 这样的离线版本相比，在线版本对安全状况的变化反应更快。一张本不该再被认可的旧证书很难清除，而在线认证服务器可以立刻让被攻破的一方的认证失效，并马上应用这些变化。这类系统的细节可能相当复杂，所以我们不深入讨论。Kerberos 就是这类在线认证服务器的一个例子 [NT94]。

[6]你可能还记得，在访问控制一章中，我们答应过要讨论在网络环境中如何用密码学来保护能力。本质上，这些 Web Cookie 就是这样的东西。用户用另一种机制证明自己的身份之后，远程系统会为这个用户创建一个别人都无法创建的密码学能力，一般用的是只有这个系统才知道的密钥。这个能力（也就是 Cookie）现在就可以传回给另一方，用于以后的授权操作。许多其他分布式系统也采用同样的基本方法。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

57.7 一些更高层的工具

有些情况下，我们可以在更高的层次上工作，达到理想的安全效果。**HTTPS**（HTTP 协议受密码学保护的版本）和 **SSH**（SSL 的一个竞争者，最常用来与远程计算机建立安全会话）就是两个很好的例子。

HTTPS

HTTP 是支撑万维网的协议，它本身没有安全功能。不过如今，大量敏感而有价值的信息都在 Web 上传送，把它们全都毫无保护地发到网络上，显然不是好主意。然而，HTTPS 并没有为 HTTP 重新实现一套安全机制，而是拿现有的 HTTP 定义，把它和 SSL/TLS 连接起来。建立安全连接的工作由 SSL 负责，包括用前面讨论的证书方法认证 Web 服务器，以及建立一把只有客户端和服务器知道的新的对称加密密钥。SSL 连接一旦建立，客户端和服务器之间后续的所有交互都使用这条安全连接。在很大程度上，HTTPS 就是通过 SSL 连接传送的 HTTP。

不过，这并不会降低 HTTPS 的重要性。实际上，它是一个很有用的范例。HTTPS 没有花上好几年去开发，也没有冒险去面对其他安全协议开发者难免会遇到的同类安全缺陷，而是直接使用一个高质量的传输安全工具，以极低的开发成本，把不安全的传输换成了高度安全的传输。

HTTPS 显然非常依赖身份认证，因为我们希望确定自己没有在和恶意网站通信。HTTPS 为此使用证书。由于 HTTPS 主要用在浏览器里，相关的证书由浏览器收集和管理。现代浏览器都预先配置了许多证书签名机构（也就是前面提到的 CA）的公钥。网站的证书要拿这些签名机构来核验，判断证书是真是假。不过要记住，证书实际告诉你的是什么（假设核验通过）：在某个时刻，签名机构认为，担保某把公钥属于某一方是个好主意。它并不意味着这一方是好是坏，不意味着对应的私钥仍然保密，甚至也不意味着证书签名机构本身是安全的、没有被攻破，无论是在它创建证书时，还是在你核验证书时。现实中，Web 证书已经因为上面所有这些原因出过问题。还要记住，HTTPS 只能为真实性担保。一个经过认证、使用 HTTPS 的网站，照样可以对你的客户端发起攻击。没错，这是一次经过认证的攻击，可如果攻击得逞，这也安慰不了你多少。

过去并不是所有浏览器都一直支持 HTTPS，通常是因为它们没有安装或配置 SSL。在这种情况下，只使用 HTTPS 的网站就无法和这样的客户端交互，因为客户端建立不了自己这一端的 SSL 套接字。Web 服务器的标准解决办法是：客户端声称自己无法使用 HTTPS 时，就退回到 HTTP。服务器这样做时，就不提供任何安全保护，就好像服务器根本没有运行 HTTPS 一样。随着浏览器和客户端机器对 HTTPS 的支持越来越普遍，出现了一股推动服务器坚持使用 HTTPS、拒绝和不能或不愿使用 HTTPS 的客户端通信的潮流。这种做法叫作 HSTS（HTTP Strict Transport Security，HTTP 严格传输安全）。HSTS 是网站的一个选项。网站如果决定支持 HSTS，那么任何客户端和它的所有交互都会受到密码学保护。不能或不愿接受 HTTPS 的客户端将不能和这样的网站交互。不少大型网站都使用 HSTS，包括 Google 的 google.com 域名，但截至 2020 年，它还远未普及。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

虽然 HTTPS 主要用来保护网页浏览，但有时也被用来保护其他类型的通信。有些开发者把 HTTP 用于与标准网页浏览相当不同的用途，对他们来说，用 HTTPS 来保护通信既自然又便宜。不过，只有当你决定用 HTTP 作为应用协议时，才能用 HTTPS 来保护你的系统，而 HTTP 主要是为支持人参与的活动而设计的。例如，HTTP 消息通常用 ASCII 编码，并带有大量为满足网页浏览需求而设计的首部。用 SSL 代替 HTTPS，你的应用可能会高效得多。你也可以用 SSH。

SSH

SSH 是 **Secure Shell**（安全外壳）的缩写，这个名字准确地描述了这个程序最初的用途。SSH 可以在 Linux 和其他 Unix 系统上使用，在一定程度上也能在 Windows 系统上使用。SSH 最初的设想是一个安全的远程 shell，但它已经发展成一个更通用的工具，可以让计算机之间进行安全的交互。这个 shell 最常用于命令行界面，但 SSH 也能支持许多其他形式的安全远程交互。例如，它可以用来保护远程 X Windows 会话。一般来说，TCP 端口都可以通过 SSH 转发，这为保护远程系统之间的交互提供了一种强大的方法。

SSH 要解决的许多问题和 SSL 遇到的一样，解决方式也往往类似。必须认证远程用户，必须建立共享的加密密钥，必须检查完整性，等等。SSH 通常依靠公钥密码学和证书来认证远程服务器。客户端往往没有自己的证书和私钥，这时可以提供用户 ID 和密码。SSH 还支持不基于证书或密码的其他身份认证方式，比如使用认证服务器（例如 Kerberos）。SSH 支持多种密码算法（既有用于身份认证的，也有用于对称加密的），客户端和服务器之间需要进行某种协商，选出一组合适的算法。

SSH 的一种典型用法，为一类常见的网络安全漏洞提供了一个很好的例子，这类漏洞叫作**中间人**（**man-in-the-middle**）攻击。当双方以为自己在直接通信，而实际上是在不知情的情况下通过一个恶意的第三方通信时，就发生了这种攻击。这个第三方能看到双方之间传递的所有消息，还能在他们不知情的情况下篡改这些消息或注入新的消息[7]。

设计良好的网络安全工具不会受许多类型的中间人攻击影响，但即使是 SSH 这样好的工具，有时也会遭受这种攻击。如果你经常用 SSH，可能亲身遇到过这样的例子。当你第一次用 SSH 登录一台从未登录过的远程机器时，你大概还没有这台远程机器的公钥。你怎么拿到它？往往不是通过证书或其他任何安全手段，而只是请远程站点把公钥发给你。然后你就有了它的公钥，接着就可以安全地认证这台机器，建立加密通信了。可是，如果你第一次尝试登录远程机器时，中间正好有一个中间人呢？那么当远程机器把公钥发给你时，中间人可以丢掉那条包含正确公钥的消息，换上一条包含自己公钥的消息。现在，你以为自己有了远程服务器的公钥，实际上拿到的却是中间人的公钥。这意味着中间人可以冒充远程服务器，而你永远也察觉不到。设计 SSH 的人很清楚这个问题，所以只要你这样使用 SSH，就会弹出一条消息，警告你有危险，并询问你是否要冒险继续。民间经验表明，大家看到这条消息时总是回答“是的，继续”，网络安全专业人员也不例外。说到这里，民间经验还表明，所有警告用户可能有不安全操作的消息总是被无视，这应该能让你明白，在自己的系统里加上这类确认消息，能带来多少安全上的好处。

[7]回想一下我们在补充中讨论的 Diffie-Hellman 密钥交换，以及那个美中不足之处。那正是中间人攻击的一个绝佳例子，因为攻击者也许能和某个正确的一方交换密钥，而不是两个正确的参与方之间交换密钥，并且不被发现。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

SSH 并不是构建在 SSL 之上的，而是一个独立的实现。因此，两者各有各的缺陷、特性和用途。在 SSH 中发现的安全缺陷，未必会对 SSL 有任何影响，反之亦然。

57.8 小结

分布式系统对现代计算至关重要，却很难保证其安全。提供分布式系统安全的基石，往往是确保连接系统各组件的不安全网络不会引入新的安全问题。组件之间发送的消息都经过加密和认证，以保护它们的私密性和完整性，并确保只有预期的用户才能使用分布式服务。SSL/TLS 这样的标准工具，以及通过 X.509 证书分发的公钥，就是用来提供这些安全服务的。远程的人类用户通常用密码来认证。大部分数据的传输都使用对称密码学，因为它比非对称密码学便宜。系统参与者在通信开始之前往往并没有共享对称密钥，所以协议的第一步通常是交换一把对称密钥。正如前面几章讨论的，密钥的保密对正确使用密码学至关重要，所以分发密钥的过程需要格外小心。Diffie-Hellman 密钥交换很常用，但它仍然需要身份认证，以确保只有预期的参与者知道这把密钥。

正如前面几章提到的，自己构建密码学解决方案很有挑战性，常常会导致安全失败。SSL/TLS、SSH 和 HTTPS 等多种工具已经攻克了许多难题，并在克服这些难题方面取得了很好的进展。我们可以用这些工具构建其他系统，避开从零构建密码学方案的许多陷阱。然而，即使是最好的安全工具，要正确使用，也取决于对工具用途和局限的理解，所以要想充分发挥它们的作用，就必须深入了解怎样把这些工具集成到自己的系统中。

记住，这些工具只提供有限的安全保证。操作系统在它直接控制的硬件上本地执行操作时能得到的那种保证，它们提供不了。因此，即使正确使用了好的认证和加密工具，系统设计者也最好仔细想清楚：执行远程站点请求的操作，或者向远程站点提供敏感信息，会带来什么后果。操作系统控制的机器边界之外发生的事，总是不确定的，因而总是有风险的。

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)

参考文献

[H14] "The Heartbleed Bug" by http://heartbleed.com/. *A web page providing a wealth of detail on this particular vulnerability in the OpenSSL implementation of the SSL/TLS protocol.*

[I12] "Information technology – Open Systems Interconnection – The Directory: Public-key and Attribute Certificate Frameworks" ITU-T, 2012. *The ITU-T document describing the format and use of an X.509 certificate. Not recommended for light bedtime reading, but here's where it's all defined.*

[NT94] "Kerberos: An authentication service for computer networks" by B. Clifford Neuman and Theodore Ts'o. IEEE Communications Magazine, Volume 32, No. 9, 1994. *An early paper on Kerberos by its main developers. There have been new versions of the system and many enhancements and bug fixes, but this paper is still a good discussion of the intricacies of the system.*

[P16] "The International PGP Home Page" http://www.pgpi.org, 2016. *A page that links to lots of useful stuff related to PGP, including downloads of free versions of the software, documentation, and discussion of issues related to it.*

© 2008–23, ARPACI-DUSSEAU (OSTEP)
© 2019–23, REIHER (SECURITY)